

GEHC Edison Round I — Master Pack

For interview with **Gajanan More** · Senior Software Architect · GE HealthCare Bengaluru

Candidate: **Bhatiya Kavya Vishnukumar** · U24AI038 · SVNIT Surat · Fri 18 Sep 2026, 10:20 IST · Teams

Resume-aligned

Architect panel

~20 min

No Cloud Run in intro

Reading order

	Section	Use for
1	Resume	What you can claim — every noun is fair game
2	Gajanan More — Findings	2-page crisp intel (incl. secure coding / Zero Trust)
3	Interviewer brief	Career timeline + how to pitch to an architect
4	Edison Interview Prep	Question bank + model answers + checklist
5	Mock 1 — Gajanan (Architect)	Timed 20-min rehearsal for this panel
6	Mock 2 — Classic Campus	OOP + coding coverage insurance
7	LogiFlow GCP End-to-End	Backup only — not on resume; if he asks hosting

Night-before rule: Memorise intro + 73.0 / 68.0 / 78.3 + one trade-off sentence. Do not mention his LinkedIn or courses. Do not volunteer Cloud Run. Soft skills: stop talking when answered; “I have not built that” is a scoring line.

Bhatiya Kavya Vishnukumar

Bachelor of Technology in Artificial Intelligence
Sardar Vallabhbhai National Institute of Technology (SVNIT), Surat

📍 Vadodara, Gujarat | kavyabhatiya44@gmail.com
GitHub | LinkedIn | LeetCode

EDUCATION

• Bachelor of Technology in Artificial Intelligence

2024 – 2028

Sardar Vallabhbhai National Institute of Technology (SVNIT), Surat

CGPA: 9.05 / 10.00 (till 4th Semester)

PROJECTS

• LogiFlow — Multimodal AI Logistics Optimization Platform

FastAPI, PostgreSQL, Next.js, Gemini API, CatBoost

- Built AI-powered multimodal logistics decision intelligence platform supporting road, rail, air, and sea route optimization.
- Developed FastAPI backend integrating traffic analysis, demand forecasting, asset utilization monitoring, delay prediction, and risk-aware route ranking pipelines.
- Built and deployed a delay-risk prediction service using HistGradientBoostingClassifier for proactive shipment risk assessment.
- Achieved **73.0% accuracy**, **68.0% delayed-shipment recall**, and **78.3% cross-validated ROC-AUC** using stratified holdout and 5-fold validation.
- Implemented multi-objective optimization and ranking pipelines using delivery time, reliability, cost, and sustainability constraints.

• AirHelp — AI Airport Assistance Platform

FastAPI, Gemma, Next.js

- Built a multilingual AI airport assistance platform supporting real-time passenger query resolution and navigation assistance.
- Developed FastAPI backend services and orchestration pipelines for speech processing, context retrieval, and response generation.
- Designed modular LLM integration workflows enabling scalable conversational assistance across multiple airport service scenarios.

• MindSettler — Counseling Session Management Platform

Django REST Framework, SQL

- Architected a role-based backend system for appointment scheduling, counselor approvals, and session lifecycle management.
- Developed RESTful APIs and normalized SQL schemas to support scalable and consistent data operations.
- Implemented authentication, authorization, and workflow automation with maintainable service-layer abstractions.

ENGINEERING RESEARCH

• SMSRF — Multi-Stakeholder Recommendation Framework

Python, NumPy, pymoo, NSGA-II, MovieLens 100K, MovieLens 1M

- Designed and implemented a multi-objective recommendation engine using NSGA-II to optimize Accuracy, Novelty, Diversity, and Serenity objectives.
- Developed reusable optimization, evaluation, and Pareto-selection pipelines for large-scale recommendation experiments.
- Built a configurable experimentation framework supporting metric computation, policy evaluation, and comparative benchmarking.
- Evaluated system behavior on **MovieLens 100K** and reproduced similar trends on **MovieLens 1M**, improving result reliability and scalability.
- Achieved up to **12.6% higher Novelty** and **20.9% higher Serenity** than NSGA-3 baselines.
- Engineered reproducible research workflows with statistical validation, robustness analysis, and automated experiment tracking.

ACHIEVEMENTS

- Selected among **Top 106 teams globally** in Google Solution Challenge 2026 for LogiFlow.
- **LeetCode**: 470+ problems solved | Max Contest Rating: 1620.
- Finalist, **Web Wonders 2025**, NIT Surat.
- Finalist, **Echelon**, conducted by ISEA and ACM SVNIT, Surat.

TECHNICAL SKILLS

Languages: C, C++, Python

Core CS: Data Structures & Algorithms, Operating Systems, Computer Networks, DBMS, OOP

Backend: FastAPI, Django REST Framework, REST APIs

AI/Optimization: Scikit-learn, Recommendation Systems, NSGA-II, pymoo

Databases: SQL, PostgreSQL, MongoDB

Data: NumPy, Pandas

Tools: Git, GitHub, Docker

Gajanan More — Findings

Public-web only · for Kavya's Edison Round I · Fri 18 Sep 2026, 10:20 IST · Teams

Senior Software Architect GE HealthCare · Bengaluru ~24 yrs TOGAF Secure coding / Zero Trust

No public interview reviews name him (Glassdoor / AmbitionBox / GFG / Reddit). Everything below about “how he will interview” is **INFERENCE FROM ROLE**, not a leak. Do **not** mention his LinkedIn or courses in the call.

~24 yrs
TOTAL EXPERIENCE

Since 2021
INSIDE GEHC

Apr 2024
SENIOR ARCHITECT

TOGAF
FOUND. + PRACTITIONER · JUN 2026

1. Confirmed facts

Item	Detail
Role now	Senior Software Architect, GE HealthCare, Bengaluru (hybrid), Apr 2024 → present
Prior at GEHC	Staff Software Architect, Aug 2021 → Apr 2024 (internal promotion)
Before GEHC	Expert Technologist — DevOps & Hybrid Cloud Management, Micro Focus (~2018–2021)
Earlier	Designer, Hewlett-Packard / HPE (cloud products) · Tech Lead, Wipro · Sr Soft Engg, Patni · Associate Consultant, i-flex (FLEXCUBE retail banking)
Education	B.E. Information Technology, Amravati University, 2005
Handle	linkedin.com/in/gajumoremumbai (Mumbai roots → Bengaluru now)

2. Cyber security — what we actually found

Not his job title. He is a *software* architect, not a “Cyber Security Architect” (that title exists at GEHC for other people).

But yes — he has invested in security learning:

- **Programming Foundations: Secure Coding** — LinkedIn Learning, issued **Sep 2022**
- **The Zero Trust Framework** — LinkedIn Learning (on his profile)
- Skill listed publicly: **secure coding**

Soft signal, not “he will grill you on pen-testing.” In a real Edison round this usually shows up as: secrets, auth, input trust, and “what if the model / API is wrong.”

IN THE ROOM

If security comes up on LogiFlow: secrets not in Git/image · JWT on planner routes · Gemini key server-side only · rate limits on heavy endpoints · “I have not done threat modelling; closest is locking keys and validating inputs.” Stop there. Do not invent OWASP theatre.

3. Other public signals (crisp)

- **TOGAF Foundation + Practitioner** (Simplilearn), posted Jun 2026 — still upskilling in enterprise architecture after ~24 years.
- **DevOps / hybrid cloud** was his Micro Focus specialty — Docker, deploy topology, environments are native language.
- **Regulated-domain DNA:** early FLEXCUBE banking + 5 years GEHC — reliability and honesty over flashy demos.
- **Tone datapoint (old recommendation):** “very nice person... knowledgeable in flexcube... ambitious and motivated.” Only soft tonal signal that exists.
- **No** public blog / talks / GitHub footprint tied to this profile. Do not fake familiarity with his work.

4. How to play a real 20-minute Round I

Likely shape INFERENCE: intro → LogiFlow as a system → “why this not that” → one cloud/security/reliability probe → why GEHC → your question.

Lead with

- Boxes and data flow, not library dump
- One rejected alternative per tech
- Honest metric bounds (73% / 0.68 / 0.783)
- Secrets + failure modes in one line each

Avoid

- “I saw you did TOGAF / Zero Trust”
- Bluffing Dockerfile / cold starts / IAM
- “I led everything” if you didn’t
- Long answers — slot is short; 60–90s then stop

IN THE ROOM

Mental model: calm senior architect who has shipped cloud platforms and sat in regulated software. He listens fully, then asks one level deeper. Clean structure + clear boundary beats buzzwords.

Ask him (pick 1): “When an intern joins, what do you most want them to understand first — the data flow, or the code they touch?” · or · “What does the reliability bar look like for a feature that can reach clinicians?”

Sources: LinkedIn snippets ([gajumoremumbai](#)) · TheOrg GEHC entry · TOGAF post Jun 2026 · Secure Coding / Zero Trust course listings on profile. Not advice to stalk — prep bias only.

Interviewer Brief — Gajanan More

Senior Software Architect · GE HealthCare Bengaluru

Panel brief

Architecture focus

Honesty statement — read this first.

There are **NO** public candidate interview reviews (Glassdoor, AmbitionBox, GFG, Reddit) that name **Gajanan More**. Nothing in this document claims to know how he personally grades, whether he is strict, or what he asked a previous candidate. His LinkedIn is authwalled, so only search-result snippets and TheOrg were readable.

Every statement in this document about *how he will interview* is a **Profile-based inference** — derived from his *role*, *tenure* and *public certifications*, not from testimony. Sections 1–3 are verifiable facts; §4 onward is inference. Treat inferences as a prep bias, never as a prediction. Do not walk in assuming you know him.

Who he is

Name	Gajanan More Confirmed
Current role	Senior Software Architect , GE HealthCare (Apr 2024 – Present), Hybrid Confirmed
Location	Bengaluru, Karnataka (LinkedIn handle <code>gajumoremumbai</code> — Mumbai roots) Confirmed
Experience	~23–24 years total in software Confirmed
Tenure at GEHC	Since Aug 2021 (~5 years): Staff Architect → Senior Architect Confirmed
Education	Bachelor of Engineering, Amravati University , 2005 Confirmed
Certification	TOGAF Enterprise Architecture Foundation + Practitioner (Simplilearn), posted Jun 2026 Confirmed
Domain history	DevOps & hybrid cloud management, cloud products, retail banking (FLEXCUBE, early career) Confirmed
Public reviews of him as an interviewer	None found. Confirmed absence

One-line mental model. You are being interviewed by a **24-year veteran architect**, not by a fresh SDE-1 running a coding screen, and not by HR. He has 5 years inside GEHC and spent the decade before that on cloud and DevOps platforms at HP/HPE and Micro Focus. He also holds TOGAF at Practitioner level — meaning he thinks in *systems*, *layers* and *trade-offs* by professional habit.

Career timeline

Period	Role / Company	What that means for you
Period	Role / Company	What that means for you
Apr 2024 – now	Senior Software Architect, GE HealthCare , Bengaluru	Owns design decisions across teams; interviews for fit into an architecture-driven org.
Aug 2021 – Apr 2024	Staff Software Architect, GE HealthCare	Promoted internally — he knows GEHC’s platforms (Edison-era digital stack) from inside.
~2018 – 2021	Expert Technologist, Micro Focus — DevOps & Hybrid Cloud Management	Containers, CI/CD, deployment topology are native language to him.
~2012 – 2021 (snippets)	Designer, Hewlett-Packard / HPE — cloud products	Long stint building cloud products, not just consuming them.
Earlier	Tech Lead, Wipro	Team leadership and delivery.
~2006 – 2010	Sr. Software Engineer / roles, Patni Computer Systems (Mumbai)	Classic services-era rigour: specs, reviews, testing.
Early career	Associate Consultant, i-flex Solutions (FLEXCUBE retail banking)	Regulated-domain software (banking) — similar mindset to regulated healthcare.
	B.E., Amravati University	

Public signals (what he chooses to show)

- **TOGAF Foundation + Practitioner, completed Jun 2026.** **Confirmed** A 24-year veteran voluntarily certifying in *enterprise architecture* in 2026 signals he is actively invested in structured architecture thinking — layers, standards, stakeholders, trade-off documentation.
- **DevOps and hybrid cloud management** was his literal job title area at Micro Focus. **Confirmed** Docker, deployment, environments and release mechanics are the opposite of exotic to him.
- **Engages with EEDP mentorship content** — liked/engaged with posts about “Best Mentor at EEDP Graduation Day at GE HealthCare”. **Confirmed engagement** He is around the early-career program that this Edison internship feeds into.
- **Older public recommendation** (banking/FLEXCUBE era): “very nice person, very knowledgeable in flexcube retail banking...highly ambitious and motivated”. **Confirmed quote, dated** It is a decade-plus old and about domain knowledge, not interviewing — but the “very nice person” framing is the only tonal datapoint that exists.
- **No public blog, talks, GitHub or conference footprint** surfaced. There is nothing to flatter him about specifically, so do not fake familiarity with his work.

What this implies for a 20-minute Edison intern Round I

Inference 1 — Structure over volume. An architect with TOGAF Practitioner is unlikely to spend a 20-minute slot on a hard competitive-programming problem. More probable: he asks you to *describe a system* and listens for whether you can name components, boundaries and data flow cleanly.

Do: answer LogiFlow as “frontend → API → delay-risk + ranking services → PostgreSQL; Gemini optional; Dockerised”. **Don’t:** start with library names. Do not volunteer Cloud Run — it is *not* on the current resume (only Docker is).

Inference 2 — “Why this and not that?” is his instinct. Architects grade trade-off reasoning, not tool lists. Expect one level of “why” after every choice you name: why FastAPI (vs Django REST), why PostgreSQL (vs MongoDB — both are on your resume), why HistGradientBoosting for delay risk, why Gemini is off the critical path.
Rule: every technology you say out loud must come with **one sentence of alternative rejected**.

Inference 3 — Docker is on your resume; Cloud Run is not. He managed hybrid cloud and DevOps for years. If you say Docker, be ready for: what is in the image, how secrets stay out of it, how the FastAPI process starts. If he asks where production runs and you know Cloud Run from the team deploy, say so honestly as *team hosting* — then stop. Do not invent cold-start numbers you cannot defend. “I have not operated that part myself” is safer than a guess.

Inference 4 — Healthcare reliability framing scores. 5 years at GEHC plus early regulated-domain (banking) work. Reliability, testing, traceability, failure behaviour and “what happens when the model is wrong” are natural values for him. Volunteer these; interns almost never do.

Inference 5 — Ownership and honesty over polish. EEDP-mentorship engagement suggests he assesses *coachability*: can this student explain what they built, admit boundaries, and take feedback. A confident “I built X, my teammate built Y” beats an inflated “I led everything.”

Inference 6 — Short slot mechanics. 20-minute slots on a 20-minute grid means he may cut you off to stay on schedule. That is scheduling, not disapproval. Keep answers 60–90 seconds and stop talking.

What NOT to do with this brief. Do not say “I saw you did TOGAF” or reference his LinkedIn — it reads as surveillance in a campus interview. Use this intel only to *choose which of your own strengths to lead with*. If he asks how you prepared, the honest answer is “I researched GE HealthCare and Edison, and prepared my projects end to end.”

How Kavya should pitch (given this specific interviewer)

Lead order — highest to lowest value for an architect

1. **LogiFlow as a system** — multimodal AI logistics (road/rail/air/sea): FastAPI, PostgreSQL, Next.js, Gemini API, CatBoost/HistGB delay-risk; multi-objective ranking on delivery time, reliability, cost, sustainability. Architecture first.
2. **Measured results, honestly bounded** — HistGradientBoostingClassifier: **73.0% accuracy, 68.0% delayed-shipment recall, 78.3% ROC-AUC**. Say why recall matters; numbers are modest but real.
3. **Multi-objective trade-offs** — same language as SMSRF (NSGA-II; up to +12.6% novelty, +20.9% serenity vs NSGA-3 on MovieLens).
4. **Docker** — on the resume Tools line. Short and precise. Do not lead with Cloud Run.
5. **Credibility markers** — Top 106 GSC 2026; CGPA 9.05/10 (till 4th sem); LeetCode **470+** (max 1620); Finalist Web Wonders 2025; Finalist Echelon (ISEA + ACM SVNIT).

The 45-second opening (use this, then stop)

YOU SAY

YOU SAY

Good morning sir, I am Kavya Bhatiya, B.Tech Artificial Intelligence at SVNIT Surat, CGPA 9.05. My main project is LogiFlow — a multimodal AI logistics platform that optimises road, rail, air and sea routes on delivery time, reliability, cost and sustainability. I built FastAPI backend pipelines with PostgreSQL and a HistGradientBoosting delay-risk service — 73 percent accuracy, 68 percent delayed-shipment recall, 78.3 percent ROC-AUC — with Next.js and Gemini on the product surface. It reached the global Top 106 in the Google Solution Challenge 2026. I also work on SMSRF, a multi-stakeholder recommender with NSGA-II, and I solve DSA regularly — 470-plus LeetCode problems, max contest rating 1620. I am here because Edison is exactly this: models that have to become systems a clinician can trust.

Three sentences to keep in your pocket

- **Trade-off sentence:** “I chose over because ; the cost of that choice was .”
- **Honesty sentence:** “I have not worked on that. The closest thing I have done is .”
- **Healthcare sentence:** “In logistics a wrong prediction costs money; in healthcare it costs trust — so the model has to be conservative, testable, and explainable to the person acting on it.”

Questions Kavya can ask him

Best two (pick from these):

1. “As an architect, when an intern joins an Edison team, what is the part of the system you most want them to actually understand — the data flow, or the code they are touching?”
2. “What does the reliability bar look like for an AI feature that reaches clinicians, compared to a normal software feature? That is the part I cannot learn from outside.”

Backups:

3. “What would a strong 8-week internship contribution look like to you — shipping one small thing end to end, or going deep on one component?”
4. “Which skill do you see early engineers underinvest in — the one that shows up two years later?”
5. “Is the Edison work closer to platform engineering or to applied ML on device and imaging data?”

Context you must not get wrong

- **Edison, one line:** GE HealthCare’s AI / digital intelligence platform — it brings AI and analytics to imaging and device data so clinicians get faster, better-supported decisions. **Confirmed from GEHC public positioning**
- **The company is GE HealthCare** — a standalone company since the 2023 spin-off from GE. Say “GE HealthCare,” not “GE.”
- **Software track base** is typically Bengaluru (JFW Technology Centre). Answer relocation with an unqualified yes. **Repeated in repo intel**
- **The framing line** that recurs in GEHC candidate write-ups: every line of code affects patients. Believe it before you say it.

Sources

- LinkedIn profile (authwalled — only search snippets readable):
[linkedin.com/in/gajumoremumbai/](https://www.linkedin.com/in/gajumoremumbai/)
- TheOrg org-chart entry, GE HealthCare:
theorg.com/org/gehealthcare/org-chart/gajanan-more
- Official interview email — role (Edison Intern Role), Round I panel (Gajanan More), 10:20 slot, Teams meeting organised by Ramana Kumar Reddy A V, GEHC South Asia Talent Acquisition.
- GEHC interview-pattern intel already collected in this repo: [GE Healthcare Preparation/](#) (senior SVNIT 2025 account, GFG SVNIT Aug 2024 EEDP tech intern, GFG on-campus, EEDP FTE write-ups, PCON EID 2024).

Repeated for clarity: no named candidate interview reviews for Gajanan More exist on any public review site. Searches across Glassdoor / AmbitionBox / GFG / Reddit returned nothing tying him to a specific interview. Any statement in this document about his interviewing behaviour is explicitly labelled **Profile-based inference** and is reasoning from his role, not evidence.

Bottom line for tomorrow, 10:20. One senior architect, roughly 15–20 minutes, almost certainly built around: *who are you* → *walk me through your best project* → *why did you build it that way* → *one CS or OOP probe* → *why GE HealthCare* → *your question*. Win it with **clean architecture narration, defensible numbers, and zero bluffing**.

Drill the answers in [Kavya_GEHC_Edison_Interview_Prep.pdf](#).

GEHC Edison Interview Prep

Bhatiya Kavya Vishnukumar · Round I desk reference

Edison Intern

GE HealthCare

Readable layout

Labels used in this document. **Confirmed** = official email or multiple candidate reports. **Repeated** = 2+ independent GFG/repo reports. **One-off** = a single write-up — useful, not a promise. **Practice** = written for rehearsal; nobody reported this question. **Inference** = reasoned from the interviewer's profile. Nothing here is a leaked question. The question bank is built from GEHC campus write-ups already in this repo plus your own resume surface area.

Slot logistics

Date & time	Friday, 18 September 2026, 10:20 IST Confirmed
Platform	Microsoft Teams — meeting organised by Ramana Kumar Reddy A V , Talent Acquisition, GEHC South Asia Confirmed
Round	Round I, panel: Gajanan More (Senior Software Architect, GEHC Bengaluru) Confirmed
Expected length	~15–20 minutes. Slots run on a 20-minute grid (10:00, 10:20, 10:40...), so the panel cannot overrun. Inference from slot grid
Role	Edison Intern Role Confirmed
Your identifiers	Reg. U24AI038 · kavyabhatiya44@gmail.com

Teams setup — do this by 10:05.

- Join from a **laptop**, wired or strong Wi-Fi. Phone hotspot charged as backup.
- Camera **on**, plain background, light in front of you not behind you. Test audio in Teams' device settings.
- Display name = **Bhatiya Kavya Vishnukumar** (matches their records; "Kavya" alone may not).
- Join at **10:17**. If nobody admits you by 10:25, wait — the prior slot may have run long. Email Ramana only after 10:35.
- Have open, minimised: your resume PDF, and one browser tab with the LogiFlow repo/deployment. Close everything else, silence notifications.
- Keep a pen and blank A4 sheet — if he asks you to explain a design, you can sketch and hold it up or narrate structurally.

Process context

- Round I here is a **single-panel technical conversation with one senior architect**, not a coding test. **Inference from panel seniority**
- Across SVNIT GEHC accounts, the technical round is **panel-dependent**: some panels drill projects and logic without live coding, others ask you to code or design with hints. **Confirmed — senior SVNIT 2025**
- GFG SVNIT Aug 2024 (EEDP tech intern) reported a ~30-minute technical: **C++ OOP** (copy constructor, virtual functions, overriding), then projects, then a light ML question ("ChatGPT uses which type of ML?"). **Confirmed**

— GFG

- Another GFG on-campus account: deep project questions **plus** OOP **plus** a Two-Sum-style problem and a list-mutation coding question. **Repeated**
- EEDP FTE write-ups: implement **runtime polymorphism live**, OSI layers, DB locks and threads, project depth. **Confirmed for FTE, heavier than intern rounds**
- HR is usually a separate later round, but Round I may include a light why GE HealthCare. Be ready; keep it to 40 seconds. **Repeated**
- **Edison, one line:** GE HealthCare's AI and digital intelligence platform — AI and analytics over imaging and device data so clinicians get faster, better-supported decisions. **Confirmed positioning**

In a 20-minute slot, the biggest risk is not difficulty — it is over-talking. Every answer 60–90 seconds, then stop. Silence after your answer is his thinking time, not your cue to keep going. If you fill it, you lose the questions where you would have scored.

Highest-probability question bank

3.1 Opening and fit

- Tell me about yourself. **Confirmed — every account** → model answer §4.1
- Why GE HealthCare? Why Edison? **Repeated** → model answer §4.2
- You are in AI branch, second year — why software/Edison and not pure research? **Practice**
- Are you comfortable with Bengaluru? **Repeated** → “Yes, no constraints.”
- What do you know about GE HealthCare's products? **Repeated** → imaging (MRI, CT, ultrasound, X-ray), patient monitoring, and Edison as the AI/analytics layer across them.

3.2 LogiFlow — expect 6–10 minutes here

- Walk me through LogiFlow. **Confirmed pattern** → model answer §4.3
- Draw / describe the architecture. What talks to what? **Inference — architect panel** → Next.js → FastAPI → multi-objective ranking + HistGB delay-risk → PostgreSQL; Gemini assist/explain only; Docker packages the service (do not volunteer Cloud Run — not on current resume).
- What did *you* personally build versus your teammates? **Repeated — ownership probe** → You founded LogiFlow: road pipeline, hybrid scoring, HistGB delay service, planner/reports, Dockerfile. Name components precisely. Never claim every mode alone.
- 73% accuracy is not high. Why should anyone trust this model? **Inference — likely, prepare it** → model answer §4.4
- Why HistGradientBoosting / CatBoost and not a neural network? **Practice** → tabular data; boosting dominates; resume lists CatBoost; delay service is HistGB with stratified holdout + 5-fold.
- How do you optimise across four objectives at once — delivery time, reliability, cost, sustainability? **Inference — his TOGAF/trade-off instinct** → normalise each objective, weight per user constraint, rank a Pareto-ish candidate set — not one fake global optimum.
- Where does Gemini actually sit? Is it making the decision? **Practice — and an important honesty test** → No. Optimiser + classifier decide; Gemini explains/assists. If Gemini fails, ranking still works.
- Why PostgreSQL and not MongoDB? **Repeated — both on resume** → LogiFlow needs joins (routes/legs/hubs/costs); Mongo fits document-shaped data elsewhere.

- Why FastAPI and not Django here, when you used Django in MindSettler? **Practice — strong differentiator** → model answer §4.6
- Docker — what is actually in the image? **Confirmed on resume Tools** → model answer §4.5
- What broke? What would you rebuild? **Repeated** → class imbalance made accuracy look fine while missing delays — moved to recall/ROC-AUC as the real metrics.
- How did you test it? **Repeated — reliability probe** → be honest about coverage; stratified holdout + 5-fold for the delay model; say what you did not automate.

3.3 AirHelp, MindSettler, SMSRF

- AirHelp — what does it do and why Gemma instead of a hosted API? **Practice** → multilingual airport passenger assistance on FastAPI + Next.js; Gemma is open-weight so it can be self-hosted, cheaper and controllable — relevant when data cannot leave the environment.
- MindSettler — how does role-based access work? **Repeated — auth probe** → Django REST with roles (counsellor / client / admin), permissions checked per endpoint, session scheduling constrained by role; SQL schema for sessions and availability.
- Your research — explain NSGA-II simply, and what “multi-stakeholder” means. **Practice** → genetic algorithm that keeps a non-dominated front (trade-offs, not one weighted answer). Multi-stakeholder = competing objectives across parties. MovieLens 100K (+ trends on 1M): up to **+12.6% novelty, +20.9% serenity vs NSGA-3**.
- Multi-objective optimisation appears in both LogiFlow and your research — is that deliberate? **Inference** → Yes, and say so: it is your through-line. Real systems have competing objectives; picking a defensible trade-off is the actual engineering.

3.4 OOP and CS fundamentals

- Copy constructor — when is it called? Shallow vs deep copy? **Confirmed — GFG SVNIT Aug 2024**
- Virtual functions — what problem do they solve? How does the vtable work? **Confirmed — GFG**
- Difference between overloading and overriding. **Confirmed — GFG**
- Implement runtime polymorphism — write it. **Confirmed for EEDP FTE, possible here** → drill in §5
- What is the *purpose* of OOP, not the four pillars? **Repeated across repo mocks** → “Keep data and the behaviour that owns it together so a change stays local. Reuse is a side effect, not the goal.”
- Abstract class vs interface. Why a virtual destructor? **Practice**
- DBMS: normalisation, indexing, ACID, what a transaction guarantees. **Repeated**
- OS: process vs thread; race condition; deadlock conditions. **Repeated**
- CN: OSI layers; TCP vs UDP; what happens when you hit a URL. **Repeated — deep OSI in FTE rounds**
- What type of ML does ChatGPT use? **One-off but literally asked at SVNIT** → “Self-supervised pretraining of a transformer on next-token prediction, then supervised fine-tuning and RLHF for alignment.”

3.5 Behavioural (short-slot versions)

- Biggest challenge in your project and how you handled it. **Repeated**
- Tell me about a disagreement in your team. **Repeated** → resolve with evidence/benchmark, not seniority.
- Strength and weakness, with reasons. **Confirmed — GFG HR** → weakness must be real plus the correction you are running.

- How do you learn something new fast? **Practice** → point at Docker packaging / HistGB + CatBoost iteration during Solution Challenge timelines.

Model answers (60–90 seconds each)

4.1 Intro

YOU SAY

YOU SAY

Good morning sir. I am Kavya Bhatiya, B.Tech in Artificial Intelligence at SVNIT Surat, CGPA 9.05 through fourth semester. My strongest project is **LogiFlow**, a multimodal AI logistics platform — it optimises road, rail, air and sea routes on delivery time, reliability, cost and sustainability instead of only the fastest path. I built FastAPI backend pipelines with PostgreSQL, a HistGradientBoosting delay-risk service — **73 percent accuracy, 68 percent delayed-shipment recall, 78.3 percent ROC-AUC** — and Next.js with Gemini on the product surface. It made the **global Top 106 in the Google Solution Challenge 2026**. I also work on SMSRF, a multi-stakeholder recommender using NSGA-II, and I keep fundamentals sharp with DSA — **470-plus** LeetCode problems, best contest rating 1620. What I enjoy most is turning a model into a system someone can actually rely on.

4.2 Why GE HealthCare / why Edison

YOU SAY

YOU SAY

Two reasons, one technical and one personal. Technically, Edison is GE HealthCare's AI and analytics platform over imaging and device data — and that is the hardest version of the problem I have been practising. In LogiFlow, if my delay prediction is wrong, someone loses money. On Edison, a clinician is making a decision with it, so the same model needs a much higher bar: it has to be reliable, testable, and explainable to the person acting on it. That constraint is what I want to learn to build under, and I cannot learn it from a hackathon. Personally — GE HealthCare's software is inside machines that touch millions of patients, and I would rather spend my first internship on code where correctness genuinely matters than on something disposable.

4.3 LogiFlow product pitch

YOU SAY

YOU SAY

The problem: when a shipment moves between cities, a planner chooses among road, rail, air and sea — options differ on cost, transit time, delay risk, and sustainability. Today that is mostly habit and phone calls. LogiFlow takes origin, destination, cargo and constraints, and returns a **ranked set of routes with trade-offs visible**, plus predicted delay risk.

Architecture: Next.js frontend, FastAPI backend, PostgreSQL for route and operational data. Inside the backend: multi-objective ranking on delivery time, reliability, cost and sustainability; and a HistGradientBoostingClassifier for delay-risk. Gemini assists on the surface — it does not decide the ranking. Docker packages the service. Global Top 106, Google Solution Challenge 2026.

4.4 Defending the delay model's metrics

YOU SAY

YOU SAY

Honest numbers first: HistGradientBoostingClassifier, **73.0% accuracy, 68.0% recall on delayed shipments, 78.3% ROC-AUC**, with stratified holdout and 5-fold validation. Accuracy is the metric I care about least here, because delays are the minority class — a model that always predicts “on time” looks fine on accuracy and is useless. The metric that matters is **recall on delays**. 68% means we catch roughly two of three real delays; 78.3% ROC-AUC says ranking beats chance across thresholds.

I use it as a **risk signal inside ranking**, not a binary verdict. To improve: more history, better weather/congestion features, calibrated probabilities, thresholds driven by the cost of a missed delay versus a false alarm.

4.5 Docker (resume claim) — hosting only if asked

YOU SAY

YOU SAY

The backend is a Docker image: Python base, locked dependencies, FastAPI app, model artefact copied in, uvicorn on the configured port. Secrets stay out of the image — runtime env, gitignored dotenv locally. Docker buys the same artefact locally and in production and keeps the filesystem honest. If asked about production hosting: the team deploys that containerised API on Cloud Run; I will not invent concurrency or min-instance numbers I did not set myself.

4.6 FastAPI vs Django — why different stacks

YOU SAY

YOU SAY

I have used both for different shapes. **MindSettler** is counselling session management — CRUD, roles, scheduling, auth — so Django REST fit: ORM, auth and permissions come built in. **LogiFlow** is computation-heavy with outbound Gemini calls — FastAPI gave async handling, Pydantic validation at the boundary, and OpenAPI for the frontend. Trade-off: with FastAPI I wrote more of my own auth/data access; worth it because the heavy work is optimisation and inference.

OOP and DSA drills for a short slot

5.1 Runtime polymorphism — write this from memory

Confirmed — asked live in EEDP rounds

```
class Shape {
public:
    virtual double area() const = 0;        // pure virtual
    virtual ~Shape() = default;            // virtual dtor: safe delete via base ptr
};
```

```
class Circle : public Shape {
    double r;
public:
    explicit Circle(double r) : r(r) {}
    double area() const override { return 3.14159 * r * r; }
};
```

```
class Rect : public Shape {
    double w, h;
public:
    Rect(double w, double h) : w(w), h(h) {}
    double area() const override { return w * h; }
};
```

```
// dispatch resolved at run time through the vtable
void print(const Shape& s) { std::cout << s.area() << "\n"; }
```

Say while writing: “`virtual` makes the call resolve on the *dynamic* type, through a vtable pointer stored in the object, so `print` works for any future shape without changing `print`. The virtual destructor is not optional — without it, deleting a `Circle` through a `Shape*` is undefined behaviour and leaks the derived part.”

5.2 Two Sum, OOP-wrapped

Confirmed — GEHC EID 2024 candidates were told to use classes, not bare functions

```
class Solution {
public:
    std::vector<int> twoSum(const std::vector<int>& nums, int target) {
        std::unordered_map<int, int> seen; // value -> index
        for (int i = 0; i < (int)nums.size(); ++i) {
            int need = target - nums[i];
            auto it = seen.find(need);
            if (it != seen.end()) return {it->second, i};
            seen[nums[i]] = i;
        }
        return {}; // no pair found
    }
};
```

Narrate: brute force is $O(n^2)$; one pass with a hash map is **$O(n)$ time, $O(n)$ space**. Then dry-run `[2, 7, 11, 15]`, target `9` out loud. Handle the “no answer” case explicitly — interviewers watch for that.

5.3 Rapid-fire answers to have ready

- **Copy constructor:** called when an object is initialised from another object of the same type (also on pass/return by value). Default is a member-wise *shallow* copy — if the class owns a raw pointer you must write a *deep* copy or you get a double free.
- **Overloading vs overriding:** overloading = same name, different parameters, resolved at *compile* time. Overriding = derived class replaces a `virtual` base method, resolved at *run* time.
- **Abstract class vs interface:** abstract class can hold state and partial implementation; an interface (all-pure-virtual class in C++) is only a contract.

- **Process vs thread:** separate address space vs shared address space within one process — which is exactly why threads need synchronisation and processes need IPC.
- **Race condition:** two threads interleaving on shared mutable state without synchronisation; fix with a mutex or by removing the shared mutability.
- **ACID:** atomicity, consistency, isolation, durability — the guarantees a transaction gives you.
- **Index:** a secondary structure (usually a B-tree) that makes reads on a column fast at the cost of slower writes and extra storage.
- **Normalisation:** remove redundancy so an update happens in one place; denormalise deliberately when read performance demands it.
- **TCP vs UDP:** ordered, reliable, connection-oriented vs fire-and-forget with lower latency.

The healthcare / reliability angle

Line 1 — the stakes shift. “In logistics, a wrong prediction costs money and you retry. In healthcare, a clinician acted on it — so the same model needs a different bar: conservative defaults, an audit trail, and an explanation the user can check.”

Line 2 — why explainability was already in my project. “I put a plain-language explanation on every LogiFlow ranking because a planner will not trust a number they cannot interrogate. That is the same reason clinical AI has to explain itself — it is not a nice-to-have, it is what makes the output usable.”

Line 3 — reliability as a habit. “The thing I would carry into a healthcare codebase is the assumption that the model *will* be wrong sometimes, so the system has to behave sensibly when it is — fall back, flag low confidence, never fail silently.”

Line 4 — if asked about regulation/compliance you do not know. “I have not worked under a regulated software process — FDA or IEC-style. I know the principle is traceability from requirement to test, and that is exactly the part of engineering I want to be trained in properly rather than guess at.”

20-minute clock plan

Clock	Phase	Your job
0:00–1:00	Greeting	Camera on, smile, “Good morning sir.” Confirm audio once, then let him lead.
1:00–2:30	Intro	§4.1, under 60 seconds. Plant LogiFlow, 73/68/78.3, Top 106, 470+ DSA . Then stop.
2:30–10:00	Project deep-dive	Architecture first, then whichever thread he pulls. Numbers only where you can defend them. Let him interrupt.
10:00–14:00	Technical probe	OOP / DBMS / OS, or a short coding or design ask. Narrate approach before syntax.
14:00–17:00	Fit / why GEHC	§4.2, 40–50 seconds. Bengaluru: unqualified yes.
17:00–19:00	Your questions	Two maximum from the interviewer brief.
19:00–20:00	Close	“Thank you sir, I enjoyed the conversation” — and mean it. Do not ask how you did.

If he runs short (10 minutes total). That is normal for a 20-minute grid with a schedule to keep. It is not a rejection signal. Make sure LogiFlow's architecture and one defensible number got said.

If he goes deeper than you can follow. Say: "I have not worked on that. The closest thing I have done is — can I tell you how I would approach it?" Then reason out loud. **Never bluff a cloud or systems answer with this interviewer.**

If he asks something you half-know. State the part you are sure of, mark the boundary explicitly, then reason forward. Half-knowledge stated honestly scores; half-knowledge stated confidently does not.

Night-before and morning checklist

Tonight (Thu 17 Sep)

- Say §4.1, §4.2, §4.3 out loud **three times each**, timed. Cut anything past 90 seconds.
- Write the **runtime polymorphism** snippet (§5.1) from memory on paper, once. Then Two Sum (§5.2).
- Re-read your own resume as an adversary: for every noun on it, can you answer one "why?" If not, prepare that one line tonight.
- Confirm your numbers: 73.0% accuracy, 68.0% delayed recall, 78.3% ROC-AUC (stratified holdout + 5-fold), +12.6% novelty, +20.9% serenity vs NSGA-3, Top 106, CGPA 9.05 (till 4th sem), LeetCode **470+**, contest max 1620, Web Wonders + Echelon finalist. **Never inflate.**
- Skim the interviewer brief once — then close it. Do not walk in trying to psychoanalyse him.
- Charge laptop and phone. Test Teams camera and mic tonight, not at 10:15.
- Lay out: resume printout, blank A4, pen, water. **Sleep by 23:30** — rested beats over-prepared.

Tomorrow morning (Fri 18 Sep, 10:20 slot)

- **09:30** — one calm read of §4 model answers. No new topics. Learning something new now only adds noise.
- **09:50** — eat something light. Do not interview hungry or over-caffeinated.
- **10:00** — close every app except Teams, your resume, and one browser tab. Silence phone, disable notifications.
- **10:10** — say the intro out loud once, at full volume, to warm your voice up.
- **10:17** — join the Teams link. Camera on, name set to **Bhatiya Kavya Vishnukumar**.
- **In the room** — sit up, look at the camera not the screen, speak slower than feels natural, and **stop talking when you have answered.**

The three ways interns lose this round — avoid all three.

1. **Inflating ownership.** "I built the whole thing" collapses under one follow-up. Name your components.
2. **Bluffing infrastructure.** He ran DevOps and hybrid cloud products for years. "I have not done that yet" is a scoring answer; a wrong confident answer is not.
3. **Talking past the answer.** In a 20-minute slot, every extra 30 seconds costs you a question you would have won.

You are ready. You have a globally recognised project (Top 106), honest metrics you can defend, SMSRF research, Docker on the resume, and 470+ DSA problems. Round I with an architect is about *how you think*. Think out loud, stay inside the resume, and stop when you have answered.

Mock 1 — Gajanan (Architect)

20 minutes · timed script · answer from memory

Mock interview

20 min

Architecture

Tone to expect: collaborative, not adversarial. He is 24 years senior — he does not need to prove anything. Danger is not hostility, it is that you will say a buzzword (“multimodal”, “multi-objective”, “HistGradientBoosting”) and he will quietly ask you to define it. Every noun on your resume is a door he can open.

How to use. Set a 20-minute timer. Read Gajanan’s line aloud (or have someone read it). Answer the green box **from memory**, out loud, then compare. Do not read ahead.

Intro to memorise (35–40 sec):

Good morning sir, I am Kavya Bhatiya, B.Tech in Artificial Intelligence at SVNIT Surat, CGPA 9.05. My main project is LogiFlow — a multimodal AI logistics platform on FastAPI, PostgreSQL and Next.js, with Gemini and a HistGradientBoosting delay-risk service — 73 percent accuracy, 68 percent delayed-shipment recall, 78.3 percent ROC-AUC. It reached Top 106 globally in the Google Solution Challenge 2026. I also do research on multi-objective recommenders using NSGA-II in SMSRF — up to 12.6 percent higher novelty and 20.9 percent higher serenity than an NSGA-3 baseline. I write C, C++ and Python, and I have solved 470-plus problems on LeetCode at a 1620 rating.

Three rules for this round. (1) Never say a number you cannot break down — he *will* ask how it was measured. (2) When you name a technology, immediately give the alternative you rejected. (3) If you do not know: “I have not built that; the closest thing I have done is ____.” Say it flatly. Architects respect a clean boundary far more than a bluffed one.

Clock

0:00–2:00	Open	Intro, why AI branch, one line on Vadodara/SVNIT if asked.
2:00–7:00	LogiFlow: product then boxes	What problem, who uses it, then the component diagram in words.
7:00–11:00	Docker / reliability / secrets	Why Docker, what is in the image, secrets, failure modes.
11:00–15:00	Delay model honesty	73% accuracy, 0.68 recall, 0.783 ROC-AUC — defend, do not oversell.
15:00–17:30	Second artefact	SMSRF NSGA-II (preferred) or AirHelp / MindSettler.
17:30–19:00	Light OOP / design	Abstraction, strategy pattern, or “how would you extend it”.
19:00–20:00	Close	Your question for him + thank you.

Script

0:00 — Opening

INTERVIEWER Good morning Kavya. I am Gajanan, I work as a software architect here at GE HealthCare in Bengaluru. Before we start — tell me about yourself, and keep it to about a minute.

YOU SAY

YOU SAY

The 35–40 second intro above. Then **stop talking**. Do not fill silence.

INTERVIEWER *You are in the AI branch — that is still fairly new at NITs. Was that your first choice, or did you land there?*

YOU SAY

YOU SAY

It was a deliberate choice, sir. I wanted the systems fundamentals — DSA, OS, computer networks, DBMS — but with the modelling layer on top, because most of what I build ends up being an ordinary backend that happens to have a model inside it. LogiFlow is exactly that: ninety percent API, database and deployment work, ten percent model.

2:00 — LogiFlow: product first, then architecture

INTERVIEWER *Let us take LogiFlow. Assume I know nothing about logistics. What is the problem, who is the user, and what do they get out of it?*

YOU SAY

YOU SAY

The user is someone who has to move freight between two cities and has more than one way to do it — road, rail, or a combination. Today that decision is made on phone calls and habit. LogiFlow takes the origin, destination, and the user's constraints — budget, delivery deadline, how much delay risk they can tolerate — and returns a ranked set of route options where each one is scored on cost, transit time, and probability of delay. The deliverable is not a map, sir, it is the trade-off: this route is cheaper but eighteen percent likelier to be late, this one costs more and is reliable. The user picks; the system makes the picking informed.

INTERVIEWER *Good. Now draw me the architecture — in words. What are the boxes, and what talks to what?*

YOU SAY

YOU SAY

Four layers. Front-end is Next.js — route comparison UI, no business logic. It talks over HTTPS to a FastAPI service, which is the only thing that touches data. Inside FastAPI: validation/auth, routing and multi-objective scoring (delivery time, reliability, cost, sustainability), and an inference module that loads the HistGradientBoosting delay-risk model. Persistence is PostgreSQL — corridors, legs, tariffs, history, planner data. Gemini is a separate outbound call for document/field assistance and explanation text — convenience path, not the ranking path. The backend is packaged with Docker (on my resume Tools line); the team hosts the containerised API in production.

INTERVIEWER *Why is Gemini a separate box in your description? What does that tell you about it?*

YOU SAY

YOU SAY

Because it is the only dependency I do not control, sir. It is a third-party call over the network with variable latency and a quota, so architecturally I treat it as untrusted and optional: it runs on the ingestion path, not on the ranking path. If Gemini is slow or down, the user can still type the shipment details manually and get a full ranked answer. The extraction is a convenience layer, so a failure there degrades the experience but does not take the product down.

INTERVIEWER *And your ranking — you have written “multi-objective ranking”. Define that for me. What exactly is multi-objective about it?*

YOU SAY

YOU SAY

Three objectives that genuinely conflict: minimise cost, minimise transit time, and minimise delay probability. The cheapest route is usually rail and usually slow; the fastest is usually road and usually costlier. Because there is no single best answer, I do not collapse them into one number and hide it. I normalise each objective onto a common scale and expose weights the user can shift — cost-sensitive versus deadline-sensitive — and the ranking recomputes. When I need a defensible set rather than a weighted sum, I look for the non-dominated options, meaning a route stays in the shortlist only if nothing else beats it on all three at once.

INTERVIEWER *So you are essentially doing Pareto filtering. Where else have you used that?*

YOU SAY

YOU SAY

In my research work, sir — SMSRF, where I used NSGA-II, a genetic algorithm, to optimise a recommender system across accuracy and novelty simultaneously. That is where the habit came from. LogiFlow uses the idea, not the evolutionary machinery — for three objectives and a bounded candidate set, running a genetic algorithm per request would be an unjustifiable latency cost. Direct scoring with dominance filtering gives the same shortlist in milliseconds.

7:00 — Docker, reliability, secrets

INTERVIEWER *Your resume lists Docker. What does containerising LogiFlow actually buy you?*

YOU SAY

YOU SAY

Same artefact locally and in production, sir. Slim Python image, dependencies locked, FastAPI starts uvicorn on the configured port, model artefact loaded once at process start. Secrets stay out of the image — env or a secret store at runtime, gitignored dotenv locally. I do not claim I personally operated every production host knob; I own the Docker packaging and the service behaviour inside the container.

INTERVIEWER Fair. Suppose traffic spikes and you run many container copies against Postgres. What goes wrong?

YOU SAY

YOU SAY

Connection exhaustion. Each instance opens a pool; N times pool size can exceed Postgres max connections under the load you wanted to serve. Keep pools small, cap instance count, and in production put a pooler in front. State stays in Postgres — any instance can serve any request.

INTERVIEWER And credentials — database password, Gemini key — where do they live?

YOU SAY

YOU SAY

Not in the image and not in the repository. Injected at runtime. Locally dotenv is gitignored. If a key might have been logged, rotate it.

Gajanan. Suppose tomorrow this has to run inside a hospital network with no internet egress. What breaks?

YOU SAY

YOU SAY

Gemini breaks first — outbound SaaS. Because it sits behind an interface on the ingestion/explain path, I swap to local extraction or manual entry and ranking stays up. FastAPI and Postgres are fine on-prem; same Docker image, different orchestrator. The delay model must be retrained on in-network data — I do not assume public logistics features transfer into a clinical setting.

11:00 — The delay model: honesty block

INTERVIEWER Your resume says the delay model gives 73 percent accuracy, 68 percent recall on delayed shipments, and 78.3 ROC-AUC. Talk me through those three numbers. Why did you print all three?

YOU SAY

YOU SAY

Because accuracy alone would be misleading, sir, and I did not want to hide behind it. The classes are imbalanced — most shipments arrive on time — so a model that predicts “on time” for everything scores respectably on accuracy while being useless for the only case the user cares about. So 73 percent accuracy is the honest headline, 68 percent recall on the delayed class is the number that says I actually catch roughly two-thirds of real delays, and 0.783 ROC-AUC is the threshold-independent measure showing the model separates the classes meaningfully better than chance, which is 0.5. Read together, they say: useful signal, not a solved problem.

Gajanan. Two-thirds. So one delayed shipment in three, you miss. Would you ship that?

YOU SAY

YOU SAY

I would ship it as an advisory, not as a guarantee — and that is exactly how it is surfaced in the product. It is presented as a delay risk score contributing to a ranking, never as “this shipment will be late”. The user is still choosing between routes, so a directionally correct risk ordering already improves the decision over having no signal. I would not ship this same model into a workflow that automatically cancelled or rebooked a shipment; that needs much higher precision and a human confirmation step.

INTERVIEWER *You said threshold-independent. So you can move the threshold. Which way would you move it, and what does it cost you?*

YOU SAY

YOU SAY

Lower it, because the errors are not symmetric. A false positive means I warned about a delay that did not happen — the user was mildly over-cautious. A false negative means the shipment was late and I said nothing, and that is the expensive one. So I would lower the decision threshold to buy recall, and I pay for it in precision with more false alarms. Where exactly to stop is not a modelling question, it is a question for whoever owns the operational cost — so I would ask for the ratio of what a missed delay costs versus a false alarm and set the threshold from that.

INTERVIEWER *Which model, and did you try others?*

YOU SAY

YOU SAY

The production model is histogram-based gradient boosting. I evaluated CatBoost as well, and I use CatBoost elsewhere. Gradient boosting on trees won because the features are tabular and mixed — distances, mode, corridor identity, seasonality — where trees handle non-linear interactions without me hand-crafting them, and histogram binning made training fast enough to iterate in minutes. I also kept a logistic regression baseline, which matters because without it I could not claim the boosting was earning its complexity. And I checked one thing specifically: that I had no feature which leaks the outcome — nothing that would only be knowable after the shipment had already arrived late.

INTERVIEWER *How did you split your data?*

YOU SAY

YOU SAY

The honest answer is that the split is the weakest part of this project. Delay data is temporal, so a random split lets the model see future shipments while predicting past ones, which flatters the metrics. The defensible approach is a time-based split — train on earlier periods, validate on the most recent — and if I were redoing this that is the first change I would make, and I would expect the numbers to come down somewhat and be more trustworthy. I would rather tell you that than quote a number I cannot stand behind.

15:00 — Second artefact (he will pick one)

INTERVIEWER *Tell me briefly about the research — SMSRF, NSGA-II. What did you actually do, and what is the result?*

YOU SAY

YOU SAY

Recommender systems are usually optimised for accuracy alone, which collapses recommendations into popular items — the user sees what everybody sees. I formulated it as a multi-objective problem and used NSGA-II, an evolutionary algorithm that maintains a population of candidate solutions and evolves a Pareto front instead of one optimum. Evaluated on MovieLens, against an NSGA-3 baseline we improved novelty by 12.6 percent and serenity by 20.9 percent. The engineering lesson I carried out of it is that when objectives conflict, publishing the front and letting the operator choose the operating point is more honest than shipping one hard-coded weighting.

INTERVIEWER *Twenty percent on a metric called serenity — is that a metric a user would feel?*

YOU SAY

YOU SAY

Not directly, sir. It is a proxy for how comfortable a recommendation set is for the user rather than how accurate it is, and I would not claim a user perceives a twenty percent improvement. What is defensible is the comparison: same dataset, same protocol, same baseline, so the relative gain is real even if the absolute human impact is not something I measured. A user study is what would be needed for that claim, and we did not run one.

INTERVIEWER *And AirHelp — one line. What is different there from LogiFlow?*

YOU SAY

YOU SAY

AirHelp is an airport assistance assistant — FastAPI backend, Next.js front-end, with Gemma as the language model for handling passenger queries. Architecturally the difference is that in AirHelp the model is the product, so latency and prompt handling are the whole engineering problem, whereas in LogiFlow the model is one component inside a decision system. MindSettler is my third one, a counselling appointment platform on Django REST with SQL — that one was where I actually learned authentication and relational schema design properly, before I moved to FastAPI.

17:30 — Light OOP / design (only if time survives)

INTERVIEWER *Small design question. Tomorrow the product team says LogiFlow must support air freight as a fourth mode. What do you change?*

YOU SAY

YOU SAY

Ideally very little in the ranking layer, and that was the intent of the design. Each transport mode sits behind a common interface — given an origin, destination and constraints, return candidate legs with cost, time and features. Rail, road and multimodal are implementations of that. Air freight becomes a fourth implementation plus its own data, and the scorer and ranker do not change because they consume the interface, not the concrete mode. Realistically, two things would still break: the delay model has no air-freight training data, so its predictions would be out of distribution and I would gate air routes to a heuristic until I had data; and air has constraints the schema does not model, like cargo weight limits, so there is a migration.

INTERVIEWER *You just described a pattern without naming it. What is it, and why not just use an if-else on mode?*

YOU SAY

YOU SAY

That is the strategy pattern, sir — behaviour selected at runtime behind a shared abstraction. The reason not to use if-else on mode is that the conditional does not stay in one place. It starts in the router, then appears in pricing, then in scoring, and adding a mode means finding every branch and hoping you found them all. With the interface, adding a mode is one new class registered in one factory, and the existing paths are not touched — which means the existing tests still mean something. It is the open-closed principle in practice: open to extension, closed to modification.

INTERVIEWER *One fundamentals question. Why would you make a method virtual in C++?*

YOU SAY

YOU SAY

So the call resolves on the actual object's type at runtime rather than the pointer's declared type. Without virtual, calling through a base pointer runs the base version — static binding — which defeats the purpose of the hierarchy. With virtual, dispatch goes through the vtable to the derived override. Two related points I keep in mind: a base class with virtual functions needs a virtual destructor, otherwise deleting a derived object through a base pointer only runs the base destructor and leaks; and virtual dispatch costs an indirection, so it is not free — in a tight numerical loop I would avoid it.

19:00 — Close

INTERVIEWER *We are nearly out of time. Do you have questions for me?*

YOU SAY

YOU SAY

Two if there is time, sir. First — given your architecture role, how much of an Edison intern’s work is inside an existing service versus greenfield? I want to know whether I should be preparing to read a large existing codebase, because that is a skill I would like to be tested on. And second, healthcare software has constraints I have never worked under — validation, regulatory, on-premise hospital deployment. How early does an intern encounter those, and is there something you would recommend I read before joining?

INTERVIEWER Good questions. Thank you Kavya, we will get back to you through the campus process.

YOU SAY

YOU SAY

Thank you sir. I enjoyed the conversation — particularly the point on air-gapped deployment; that is something I had not thought through before.

Score this mock

Rate each area /10. Pass bar: 7+ average, and nothing below 5.

Intro	Under 45 s? Named LogiFlow with one proof and one number? Stopped cleanly?	
Product framing	Explained LogiFlow with zero framework names? User + problem + outcome?	
Architecture	Named the boxes and what talks to what? Separated critical path from convenience path?	
Trade-offs	Every technology came with the rejected alternative — FastAPI vs Django, HistGB vs baseline, Postgres vs Mongo?	
Reliability	Docker image, secrets, connection limits, no-egress scenario — answered without panic?	
Metric honesty	Explained <i>why</i> three metrics? Owned the temporal-split weakness with a fix?	
Design / OOP	Strategy pattern reasoning? Virtual + virtual destructor without prompting?	
Close	Question aimed at his world, not at perks?	

Pass bar for tomorrow. You never said “multi-objective” or “multimodal” without immediately defining it in one plain sentence. You quoted 73 / 0.68 / 0.783 and could explain why each exists. You said “I have not built that” at least once without flinching. You did not claim the delay model was production-grade.

Three killers to avoid. (1) Reciting the stack as a list — he does not care that it is FastAPI, he cares why not Django. (2) Saying “accuracy is 73 percent” and stopping. (3) Inflating the research — it is a strong undergraduate contribution, do not call it state of the art.

Next: Kavya_GEHC_Mock2_Classic_Campus.pdf — same timer, C++ OOP and live coding, no notes.

Mock 2 — Classic Campus Round

C++ OOP · live coding · LogiFlow · timed

Mock interview

OOP + coding

Difference from Mock 1: Mock 1 assumes an architect who drills design. This one assumes a conventional panel that wants to confirm you know your language and can write correct code with your mouth moving. Both are plausible for a 20-minute slot at 10:20.

How to use. 25-minute timer. Read the panel line aloud, answer from memory. For the coding sections, **write the code by hand on paper** while narrating — do not type, and do not look at the printed solution until you have finished your own.

Intro to memorise (35–40 sec):

Good morning. I am Kavya Bhatiya, B.Tech in Artificial Intelligence at SVNIT Surat, CGPA 9.05, from Vadodara. I code in C, C++ and Python — 470-plus problems on LeetCode, max contest rating 1620. My strongest project is LogiFlow, a multimodal AI logistics platform on FastAPI, PostgreSQL and Next.js with a HistGradientBoosting delay-risk service, which placed in the Global Top 106 of the Google Solution Challenge 2026. I also work on multi-objective recommender research using NSGA-II. My coursework covers DSA, OOP, operating systems, computer networks and DBMS.

Rules. (1) Coding: restate the problem, state brute force with complexity, then optimise — never jump straight to the hash map silently. (2) OOP: give the one-line definition, then *one* concrete example, then stop. (3) Never say “I don’t remember” about a core subject — reason it out loud from first principles instead.

Clock

0:00–2:00	Intro	Self-introduction, subjects you are comfortable with.
2:00–8:00	C++ OOP	Copy constructor (deep vs shallow), virtual, polymorphism, possible dry-run.
8:00–14:00	LogiFlow deep dive	Your contribution, stack justification, model metrics, what broke.
14:00–20:00	Coding	Two Sum wrapped in a class, then an in-place array mutation.
20:00–23:00	DBMS / OS	Normalisation, index, ACID; process vs thread, deadlock.
23:00–25:00	Why GEHC + close	Motivation, location, your question.

Script

0:00 — Intro

INTERVIEWER Good morning. Please introduce yourself.

YOU SAY

YOU SAY

The 35–40 second intro above. End with the subject list — it invites them to pick, which means you control the menu.

INTERVIEWER Which subjects are you most comfortable in?

YOU SAY

YOU SAY

Data structures and algorithms first, then object-oriented programming in C++, then DBMS. I am comfortable with operating systems and networks at the concept level — I can reason about processes, scheduling and memory, and about the TCP and HTTP layers as I use them in my backends — but DSA and OOP are where I would rate myself strongest.

2:00 — C++ OOP fundamentals

INTERVIEWER What is a copy constructor, and when is it called?

YOU SAY

YOU SAY

A copy constructor creates a new object as a copy of an existing object of the same class. Its signature takes a reference to a const object of the class — `ClassName(const ClassName& other)`. It is called in four situations: when I initialise one object from another, when an object is passed to a function by value, when an object is returned by value, and when the compiler needs a temporary copy. The parameter must be a reference, because if it were taken by value, passing the argument would itself require a copy, which would call the copy constructor again — infinite recursion. The compiler will not accept it.

INTERVIEWER And if you do not write one, what does the compiler give you?

YOU SAY

YOU SAY

A default copy constructor that does a member-wise copy — a shallow copy. For a class holding only value members that is correct and I should not write my own. The problem is a class that owns a raw pointer: the default copies the pointer, not what it points to, so two objects now point at the same memory. Both destructors run `delete` on it, and that is a double free, plus any write through one object silently corrupts the other. That is when I need a deep copy — allocate my own buffer and copy the contents.

INTERVIEWER Show me. Write a class where the default copy would be wrong.

```
class Buffer {
    int* data;
    size_t n;
public:
    explicit Buffer(size_t n) : data(new int[n]), n(n) {}

    // Deep copy: allocate our own storage, then copy contents.
    Buffer(const Buffer& o) : data(new int[o.n]), n(o.n) {
        std::copy(o.data, o.data + o.n, data);
    }
};
```

```

    }

    // Copy assignment: guard self-assignment, free old, then copy.
    Buffer& operator=(const Buffer& o) {
        if (this == &o) return *this;
        int* tmp = new int[o.n];           // allocate BEFORE delete
        std::copy(o.data, o.data + o.n, tmp); // so a throw leaves us valid
        delete[] data;
        data = tmp;
        n = o.n;
        return *this;
    }

    ~Buffer() { delete[] data; }
};

```

YOU SAY

YOU SAY

Three things I would say while writing this, sir. One: if I need a custom destructor, I almost certainly need a custom copy constructor and copy assignment too — that is the Rule of Three. Two: in the assignment operator I allocate the new buffer before deleting the old one, so if allocation throws, the object is still in a valid state. Three: in modern C++ the better answer is to not own raw memory at all — use `std::vector` or a smart pointer and the compiler-generated copy becomes correct, which is the Rule of Zero. I wrote it manually because you asked me to show where the default breaks.

INTERVIEWER *What is polymorphism? Types?*

YOU SAY

YOU SAY

One interface, many behaviours. Two kinds. Compile-time polymorphism, which is function overloading, operator overloading and templates — the compiler resolves which version to call from the argument types, so it costs nothing at runtime. And runtime polymorphism, which is achieved through virtual functions and inheritance — I hold a base class pointer or reference, and the call dispatches to the derived override based on the object's actual type at run time, through the vtable.

INTERVIEWER *Explain virtual with an example, and tell me what happens without it.*

```

struct Shape {
    virtual double area() const = 0;    // pure virtual -> Shape is abstract
    virtual ~Shape() = default;        // MUST be virtual: base ptr delete
};

struct Circle : Shape {
    double r;
    explicit Circle(double r) : r(r) {}
    double area() const override { return 3.14159 * r * r; }
};

```

```

struct Rect : Shape {
    double w, h;
    Rect(double w, double h) : w(w), h(h) {}
    double area() const override { return w * h; }
};

// The point: this loop never mentions Circle or Rect.
double total(const std::vector<Shape*>& v) {
    double s = 0;
    for (const Shape* p : v) s += p->area();    // virtual dispatch
    return s;
}

```

YOU SAY

YOU SAY

Without `virtual` on `area`, the call through `Shape*` would bind statically to `Shape::area` — every shape would report the base version, and the whole hierarchy would be pointless. And the virtual destructor is not optional: if I delete a `Circle` through a `Shape*` and the destructor is non-virtual, only the base destructor runs, so any resource the derived class owns leaks. The practical value of this is in the `total` function — it does not name a single concrete shape, so adding a triangle tomorrow does not touch it. I used exactly this idea in LogiFlow: transport modes sit behind a common interface so the ranking code does not branch on mode.

INTERVIEWER *One dry-run. If I call a virtual function from inside a base class constructor, which version runs?*

YOU SAY

YOU SAY

The base class version, not the override. During base construction the derived part of the object does not exist yet, so the vtable pointer still refers to the base class — dispatch cannot reach an override that would touch uninitialised derived members. That is why calling virtual functions from constructors and destructors is a known trap; the usual fix is two-phase initialisation, construct then call an `init` that can safely dispatch.

8:00 — LogiFlow deep dive

INTERVIEWER *Tell me about your best project in detail.*

YOU SAY

YOU SAY

LogiFlow. Freight between two cities usually has more than one viable way — road, rail, air, sea, or a combination — and the choice is a trade-off among delivery time, reliability, cost and sustainability. LogiFlow takes the request and constraints and returns ranked options so the planner sees the trade-offs. Backend is FastAPI with PostgreSQL; front-end is Next.js; Gemini assists on the product surface; delay risk comes from a HistGradientBoostingClassifier — 73 percent accuracy, 68 percent delayed-shipment recall, 78.3 percent ROC-AUC. Docker is how we package the service. It reached the Global Top 106 in the Google Solution Challenge 2026.

INTERVIEWER *What is your specific contribution, and what did you find hardest?*

YOU SAY

YOU SAY

I worked on the backend and the model side — the FastAPI service, the Postgres schema for corridors and shipment history, the scoring and ranking logic, and training and integrating the delay classifier. The hardest part was not the model, it was making the model's output mean something in the product. My first version returned a raw probability, and a probability on its own is not a decision — so I had to normalise cost, time and delay risk onto a comparable scale and expose weights so a deadline-driven user and a cost-driven user get different orderings from the same data. Getting that normalisation to not be arbitrary took the most iterations.

INTERVIEWER *You said multimodal. What does that mean here?*

YOU SAY

YOU SAY

Two things, and I should separate them. In the logistics sense, multimodal means a shipment can combine transport modes — rail for the long haul, road for first and last mile. In the input sense, the system accepts more than text: a user can upload a document or image of shipment details and we use Gemini to extract structured fields instead of making them type it in. That extraction sits on the ingestion path only — if it fails, manual entry still works and the ranking is unaffected.

INTERVIEWER *And the delay model — what accuracy did you get?*

YOU SAY

You say. Seventy-three percent accuracy, but I would not want to give you only that number. The classes are imbalanced — most shipments are on time — so accuracy flatters any model that just predicts “on time”. The number that matters is recall on the delayed class, which is sixty-eight percent, meaning I catch about two-thirds of genuine delays, and ROC-AUC of 0.783, which shows real separation against 0.5 for random. The model is histogram-based gradient boosting; I benchmarked it against a logistic regression baseline so I could tell whether the extra complexity was earning anything. It is shipped as advisory risk feeding a ranking, never as a promise that a shipment will be late.

INTERVIEWER Any bug you remember debugging?

YOU SAY

YOU SAY

Yes — a container worked locally and failed after deploy. The model artefact path existed on my machine but was not copied into the image, so the process started and prediction requests failed. Fix: copy the artefact in the Docker build, load once at startup, and fail loud at boot. Lesson: “works on my machine” is about my filesystem, not my code.

14:00 — Coding

INTERVIEWER Simple one. Given an array of integers and a target, return the indices of the two numbers that add up to the target. Since we have been talking OOP, wrap it in a class.

YOU SAY

YOU SAY

Restating: one array, one target, return the two indices, and I will assume exactly one valid pair unless you tell me otherwise. Brute force is two nested loops — $O(n^2)$ time, $O(1)$ space. I can do better by trading space for time: as I walk the array once, for each element I compute what value I still need, and check whether I have already seen it. A hash map from value to index gives me that lookup in $O(1)$ average, so the whole thing is $O(n)$ time and $O(n)$ space. Let me write it.

```
class TwoSum {
public:
    // O(n) time, O(n) space. Single pass.
    std::vector<int> solve(const std::vector<int>& nums, int target) const {
        std::unordered_map<int,int> seen; // value -> index
        for (int i = 0; i < (int)nums.size(); ++i) {
            int need = target - nums[i];
            auto it = seen.find(need);
            if (it != seen.end())
                return { it->second, i }; // earlier index first
            seen[nums[i]] = i; // insert AFTER lookup
        }
        return {}; // no valid pair
    }
};
```

YOU SAY

YOU SAY

Three points I would narrate as I write. First, I insert into the map *after* the lookup, not before — if I inserted first, an element could pair with itself when `target` is exactly twice that element. Second, the map stores value to index, not index to value, because I search by value. Third, duplicates are safe: if the same value appears twice, the second occurrence finds the first in the map before it overwrites it.

Complexity is $O(n)$ average; worst case $O(n^2)$ in a pathological hash-collision scenario, which is theoretical here.

INTERVIEWER Dry-run it for me on `[2, 7, 11, 15]`, target `9`.

Say this out loud, index by index:

`i=0`, `nums[0]=2`, `need = 9-2 = 7`. Map is empty, so no hit. Insert `{2 -> 0}`.

`i=1`, `nums[1]=7`, `need = 9-7 = 2`. Map has `2` at index `0`. Hit — return `{0, 1}`.

Second case to volunteer: `[3, 3]`, target `6`. `i=0`: need `3`, map empty, insert `{3 -> 0}`. `i=1`: need `3`, found at `0`, return `{0, 1}`. This is the case that breaks a naive insert-first version.

Third: `[1, 2]`, target `100` — loop finishes, return empty vector.

INTERVIEWER Now a follow-up. Same array, but suppose it is already sorted and I do not want extra space.

YOU SAY

YOU SAY

Then two pointers, sir — one at each end. If the sum is too large I move the right pointer left, because everything to its right is larger and cannot help; if the sum is too small I move the left pointer right. $O(n)$ time, $O(1)$ space. It only works because sorted order tells me which direction reduces the sum, which is exactly the information the unsorted version lacks — and that is why the unsorted version has to pay memory instead.

```
std::vector<int> twoSumSorted(const std::vector<int>& a, int target) {
    int l = 0, r = (int)a.size() - 1;
    while (l < r) {
        long long s = (long long)a[l] + a[r];    // widen: avoid overflow
        if (s == target) return { l, r };
        if (s < target) ++l; else --r;
    }
    return {};
}
```

INTERVIEWER One more, in-place. Move all zeroes in an array to the end, keeping the order of the non-zero elements. No extra array.

YOU SAY

YOU SAY

Two-pointer write-index pattern. I keep a write position starting at zero, scan with a read pointer, and every time I see a non-zero element I swap it into the write position and advance the write pointer. When the scan finishes, everything before the write pointer is the non-zero elements in their original order, and everything after is zeroes. One pass, $O(n)$ time, $O(1)$ extra space, and the swap version writes each element at most once.

```
void moveZeroes(std::vector<int>& nums) {
    int w = 0; // next write position
    for (int r = 0; r < (int)nums.size(); ++r) {
        if (nums[r] != 0)
            std::swap(nums[w++], nums[r]); // w never overtakes r
    }
}
// [0,1,0,3,12] -> r=0 zero skip | r=1 swap(w0,r1) -> [1,0,0,3,12], w=1
//               -> r=2 zero skip | r=3 swap(w1,r3) -> [1,3,0,0,12], w=2
//               -> r=4 swap(w2,r4) -> [1,3,12,0,0], w=3
```

20:00 — DBMS and OS

INTERVIEWER *You used Postgres. What is normalisation and why do it?*

YOU SAY

YOU SAY

Organising a schema so each fact is stored once, by removing redundancy through decomposition into related tables with keys. First normal form means atomic column values, second means no partial dependency on part of a composite key, third means no transitive dependency — non-key attributes depend on the key and nothing else. The reason is update anomalies: if a carrier's name is duplicated across ten thousand shipment rows, one rename means ten thousand updates and any missed row is now a contradiction. In LogiFlow, corridors, carriers and shipments are separate tables joined by foreign keys for exactly that reason. The honest counterpoint is that normalisation costs joins, so for a read-heavy analytics query I would deliberately denormalise or keep a derived table — normalise by default, denormalise with a measured reason.

INTERVIEWER *What is an index, and why not index every column?*

YOU SAY

YOU SAY

An index is a separate structure, typically a B-tree, that lets the engine find rows matching a predicate without scanning the whole table — turning a linear scan into a logarithmic lookup. You do not index everything because every index must be updated on insert, update and delete, so writes get slower and storage grows. Indexes also do not help low-cardinality columns much — an index on a boolean rarely beats a scan. I index what appears in `WHERE` and `JOIN` conditions on large tables and nothing else.

INTERVIEWER *ACID — one line each.*

YOU SAY

YOU SAY

Atomicity: a transaction is all or nothing, no partial effect survives. Consistency: it moves the database from one valid state to another, respecting constraints. Isolation: concurrent transactions do not observe each other's intermediate state — how strictly is the isolation level. Durability: once committed, it survives a crash, which is what the write-ahead log guarantees. The one I would emphasise for a logistics system is atomicity — a tariff update that half-applies is worse than one that fails outright.

INTERVIEWER *Process versus thread.*

YOU SAY

YOU SAY

A process has its own address space; threads live inside a process and share its address space, code and data while each keeping its own stack, registers and program counter. Consequences: threads communicate cheaply through shared memory but can corrupt each other, so they need synchronisation, and one thread crashing can take the process down. Processes are isolated and safer, and communication costs an IPC mechanism. Context switching between threads is cheaper because the memory map does not change.

INTERVIEWER *Deadlock — conditions, and how do you handle it?*

YOU SAY

YOU SAY

Four conditions must hold simultaneously: mutual exclusion, hold and wait, no preemption, and circular wait. Break any one and deadlock cannot occur. In practice the approaches are prevention — for instance imposing a global ordering on lock acquisition so a cycle is impossible, which is the one I would actually use; avoidance, as in Banker's algorithm, when resource needs are known in advance; or detection and recovery, where you build a wait-for graph, look for a cycle, and kill or roll back a victim. Databases usually detect and abort one transaction, which is why application code must be prepared to retry.

23:00 — Why GE HealthCare, and close

INTERVIEWER *Why GE HealthCare? And why the Edison internship specifically?*

YOU SAY

YOU SAY

Two honest reasons. The first is the nature of the software. Almost everything I have built has been a model sitting inside a real system, and the part I find genuinely interesting is the system — the API, the schema, the deployment, what happens when the model is wrong. Edison is exactly that at a scale I cannot reach on my own: AI that has to run inside clinical workflows, where being wrong has a cost, so there is no room for a model that only looks good on a slide. That constraint is the reason I want to work there rather than a place where the model is the demo. The second is that I want to be forced to raise my standards on validation and reliability. On LogiFlow I have already run into the difference between a metric and a decision, and I would like to learn how that is done properly, by people who have done it for twenty years.

INTERVIEWER *You have research and hackathon wins. Would you not rather do a PhD or a startup?*

YOU SAY

YOU SAY

Research taught me how to be rigorous about comparisons, and I want to keep that habit, but what I actually enjoy is shipping — and I would rather learn engineering discipline inside a team that has real users and real constraints than optimise a benchmark on my own. If I pursue research later it will be better research for having built production systems first. Right now, an internship where I own a real component is the highest-value thing I can do.

INTERVIEWER *You are from Vadodara, studying in Surat. Any issue with Bengaluru?*

YOU SAY

YOU SAY

None at all. I am happy to relocate — no constraint on my side.

INTERVIEWER *Any questions for us?*

YOU SAY

YOU SAY

Yes — what does the first month look like for an Edison intern? Specifically, whether I would be reading an existing codebase or starting something new, because if it is the former I would like to prepare for that deliberately. And is there a language or framework the team uses that I should get ahead on before joining?

INTERVIEWER *Thank you Kavya.*

YOU SAY

YOU SAY

Thank you sir, I appreciated the questions.

Score this mock

Rate each area /10. Pass bar: 7+ average, and nothing below 5.

Intro	Under 45 s? LeetCode + LogiFlow + one number? Ended with subject menu?	–
C++ OOP	Copy ctor: const-ref reason + shallow/deep + Rule of Three, unprompted?	–
Virtual	Static vs dynamic binding explained? Virtual destructor mentioned yourself?	–
Project	Own contribution stated honestly? Volunteered 0.68 recall before being asked?	–
Coding	Brute force + complexity stated <i>before</i> optimising? Code compiled in your head?	–
Dry run	Walked [2, 7, 11, 15] index by index? Volunteered [3, 3] edge case?	–
DBMS / OS	Normalisation with the anomaly reason? Deadlock: all four conditions?	–
Why GEHC	Specific to Edison and clinical constraint, not generic “great company”?	–

Pass bar for tomorrow. You wrote Two Sum without a syntax hesitation and caught the insert-after-lookup point yourself. You said “Rule of Three” and “virtual destructor” before being prompted. You gave the recall number without being pushed. Your “why GEHC” mentioned a constraint, not a compliment.

Five things that lose this round. (1) Jumping to the hash map with no brute-force baseline. (2) Reciting “abstraction, encapsulation, inheritance, polymorphism” as a list with no example. (3) Saying accuracy 73 percent and stopping. (4) “I forgot” on deadlock or ACID — reason it out instead. (5) A generic “why GEHC” about brand and culture.

Last 10 minutes before the slot. Re-read only three things: your 40-second intro, the 73 / 0.68 / 0.783 explanation, and the copy constructor answer. Nothing else.

LogiFlow on GCP

Layman → every Cloud Run knob → full request & deploy path

team-3mo

2 CPU · 2 GiB

min 1 · max 3

asia-south1

Production Cloud Run — memorize first

2 vCPU

CPU

2 GiB

MEMORY

min 1

ALWAYS WARM

max 3

COST CEILING

40

CONCURRENCY

300 s

TIMEOUT

Service logiflow-api · **Region** asia-south1 · **Profile** team-3mo · **Workers**

COMPOSE_PARALLEL_WORKERS=8 · **Warmup** /api/warm-backend (load, focus, every 3 min)

One line: FastAPI on Cloud Run at 2 CPU / 2 GiB, min 1 max 3, concurrency 40, 300s timeout — sized after Render's **512 MB** OOM'd on compose and rail CSVs.

How to read this doc

Part B = layman. Part C = every knob with LogiFlow's real value. Part D–E = live request + deploy. Part F = interview answers.

Part B — Layman start

What problem are we solving?

LAYMAN LogiFlow's brain is a Python FastAPI app. Judges and users need it online with enough RAM for heavy compose jobs. GCP Cloud Run is the rented computer that runs our Docker image and gives an HTTPS URL.

Two boxes that matter

1. **Frontend (Next.js)** — on **Vercel**. Browser → Vercel → API proxy.
2. **Backend (FastAPI)** — on **GCP Cloud Run** in `asia-south1`. This PDF is about that box.

LAYMAN A Docker image is a sealed lunchbox: Python 3.11 + code + libraries. Cloud Run opens it and runs `uvicorn` on port 8080. Same lunchbox locally and in prod.

Cloud Run is not the database. Postgres / Supabase / Redis live elsewhere. Cloud Run is stateless compute — instances can die; durable state stays outside.

Part C — Every Cloud Run knob (LogiFlow values)

C1. Memory — why 2 GiB?

LAYMAN RAM is working space for one container. Need more than the limit → platform kills the process = **OOM**.

Host	RAM	What happened
Render free (old)	512 MB	Rail CSV / compose → OOM crashes
Cloud Run free	1 GiB	Student budget; still tight for heavy compose
Cloud Run team-3mo (prod)	2 GiB	Headroom for workers + model + in-process caches

Why LogiFlow: Compose runs modes/legs in parallel; rail data + sklearn + geometry need RAM. Also set `RAIL_PRELOAD_ON_STARTUP=false` so huge CSVs are not loaded at boot.

Q. Why 2 GiB and not 512 MB or 4 GiB?

YOU SAY

We OOM'd on Render at 512 MB. 2 GiB was enough with 8 compose workers. 4 GiB would cost more without a measured need — we sized up from a real crash.

C2. CPU — why 2 vCPU?

LAYMAN CPU = cores the container can use. Parallel Python work wants more than one.

Why LogiFlow: `COMPOSE_PARALLEL_WORKERS=8` and multi-mode optimize (road/rail/air/water) need 2 vCPU. Free profile uses 1 CPU to save money.

C3. Min instances — why 1 (not 0)?

LAYMAN Min instances = containers kept running when idle. Min 0 = scale to zero (cheap, cold start). Min 1 = always warm (pay for latency).

Profile	Min	User experience
free	0	Idle ≈ \$0; first hit after sleep pays 5–15 s cold start
team-3mo (prod)	1	Always warm — demos / compose start immediately

Why LogiFlow: Solution Challenge demos cannot eat a 10s wake. We buy warmth from the 3-month credit budget. Frontend still hits `/api/warm-backend` on load, focus, and every 3 minutes.

Q. What is the cost of min instances = 1?

YOU SAY

We give up scale-to-zero billing to buy predictable latency. If budget forced us, drop to free profile (min 0, 1 GiB) and lean on warmup pings.

C4. Max instances — why 3?

LAYMAN Hard ceiling on container count — stops spikes/abuse from multiplying bill and DB connections.

Why LogiFlow: Rate limits ~8/min/IP; optimize semaphore ~5. Max 3 caps worst-case spend and connection fan-out.

C5. Concurrency — why 40?

LAYMAN How many HTTP requests one container handles at once. Higher = fewer containers (cheaper); too high = CPU queueing and latency cliffs.

Why LogiFlow: Deploy sets `--concurrency 40`. FastAPI is async for I/O, but scoring/compose burn CPU. Tune from p95 metrics — not folklore.

C6. Timeout — why 300 seconds?

Layer	Budget
Vercel <code>/api/compose</code>	90 s <code>maxDuration</code>
Cloud Run request timeout	300 s
Per-mode optimize futures	~30 s each

Why LogiFlow: Hybrid compose can run 60–90+ seconds. Short host timeouts failed us; 300s matches the product.

C7. Profiles side by side

Knob	team-3mo (prod)	free	Old Render
Min instances	1	0	n/a (sleep)
Max instances	3	lower	1 box
CPU	2	1	shared tiny
Memory	2 GiB	1 GiB	512 MB
Timeout	300 s	300 s	shorter / sleep
Cold start	≈0 (warm)	5–15 s	30–90 s after sleep

Interview rule. If you say “Cloud Run,” immediately name **2 CPU / 2 GiB / min 1 / max 3 / concurrency 40 / 300s**.

Part D — End-to-end live request

Happy path (optimize / rank)

1. Browser loads Next.js on **Vercel**.
2. UI calls same-origin `/api/backend/...` (no CORS pain; hides raw Cloud Run URL).
3. Vercel proxies to `https://logiflow-api-....run.app`.
4. Cloud Run TLS edge terminates HTTPS; container sees HTTP on **8080**.
5. With **min instances = 1**, a warm instance usually exists.

6. FastAPI validates JWT/input; slowapi may rate-limit (~8/min/IP).
7. Pipelines run. Cache-aside: memory → Redis → Supabase.
8. JSON returns; UI renders ranked options / map.

Compose path (why 2 GiB + 300s exist)

1. UI hits Vercel `POST /api/compose` (90s) or stream endpoints.
2. Cloud Run `RouteComposer` fans out legs with **8** parallel workers.
3. Leg cache: L1 memory → L2 Redis (~6h TTL) → L3 Supabase.
4. Redis down → fall back to memory/Supabase/recompute — do not 500 the whole request.

Part E — End-to-end deploy

LAYMAN Deploy = freeze code into an image → store it → tell Cloud Run to run that image with these knobs → keep old revision for rollback.

1. `gcloud builds submit backend` → image `gcr.io/.../logiflow-api`.
2. `team-3mo` ⇒ min 1, max 3, CPU 2, memory **2Gi**.
3. `gcloud run deploy` with `--memory 2Gi --cpu 2 --timeout 300 --concurrency 40 --min-instances 1 --max-instances 3 --port 8080`.
4. Env from `backend/.env` (never committed): Gemini, TomTom, Supabase, JWT, Redis, Google client id...
5. Runtime: `COMPOSE_PARALLEL_WORKERS=8`, `RAIL_PRELOAD_ON_STARTUP=false`.
6. Smoke: `curl .../health` → `{"status":"ok"}`.

Identity split

Deploy SA (`logiflow-deploy`) builds/pushes/deloys. Runtime SA is the running service. User Google Sign-In is a third identity — not the deploy key.

Part F — Interview scripts

Q. Walk me through LogiFlow on GCP.

YOU SAY

Frontend on Vercel; FastAPI on Cloud Run in asia-south1. Production is 2 vCPU, 2 GiB, min 1, max 3, concurrency 40, 300s timeout. We left Render's 512 MB after OOM on compose/rail. Redis and Supabase hold shared cache and durable data; Cloud Run stays stateless.

Q. Explain min / max / memory / CPU for your service.

YOU SAY

Min 1 keeps one warm container for judges. Max 3 caps cost and connections. 2 GiB is the room compose needed after 512 MB failed. 2 CPU matches parallel pipelines. Concurrency 40 multiplexes requests until CPU latency says otherwise.

Q. Why not Kubernetes?

YOU SAY

One primary API container. Cloud Run gave HTTPS, revisions, and the exact CPU/RAM knobs. GKE would add cluster overhead we did not earn.

Q. Redis down?

YOU SAY

Fall back to memory and Supabase, then recompute. Redis speeds repeats across instances — it is not the source of truth for tariffs.

Do not invent: GKE, “Redis is the DB,” “512 MB was fine,” “cold starts don’t matter.”
Do say: Render 512 MB OOM → Cloud Run 2 GiB; min 1 for demos; 300s for compose; Vercel proxies; warmup every 3 min.

Part G — Pocket card

Stack split	Vercel (UI) + Cloud Run (API)
Region / profile	asia-south1 / team-3mo
CPU / RAM	2 vCPU / 2 GiB
Min / Max	1 / 3
Concurrency / timeout	40 / 300 s (Vercel compose 90 s)
Workers / rail boot	8 / preload false
Rate limit / optimize guard	~8/min/IP · semaphore ~5 + 1h cache
Cache tiers	memory → Redis → Supabase
Origin story	Render 512 MB OOM → Cloud Run 2 GiB + min 1